

Basic Logical Formal Language

Comprehensive Notes on Predicate Logic and Formal Semantics

1. From Propositional Logic to Predicate Logic

Propositional logic treats entire sentences as indivisible units. If we symbolize “Rina laughs” as P and “Amit sings” as Q , then propositional logic allows us to build complex expressions such as $P \wedge Q$ or $P \rightarrow Q$. However, this system fails to represent the internal semantic structure of sentences. It cannot distinguish between “Rina loves Amit” and “Amit loves Rina,” nor can it represent quantificational statements such as “Every student passed.”

Natural language requires a richer system capable of representing internal argument structure and variable binding. Predicate logic (first-order logic) introduces predicates, variables, constants, and quantifiers, enabling us to model the relational structure of language. It thus forms the foundation of truth-conditional semantics.

2. Syntax of First-Order Predicate Logic ()

In order to use predicate logic as a formal language for representing meaning, we must first define its syntax with complete precision. Unlike natural language, whose grammatical rules are often implicit and sometimes ambiguous, a formal language is defined by explicit formation rules that determine exactly which strings of symbols count as well-formed expressions. This syntactic precision is essential because semantic interpretation is defined recursively over syntactic structure. If we do not clearly define what counts as a legitimate formula, we cannot systematically define its truth conditions.

A first-order language $\mathcal{L}$ consists of several distinct categories of symbols. These are divided into logical symbols and non-logical symbols. Logical symbols are shared across interpretations and contribute structural meaning; non-logical symbols receive interpretation relative to a model.

2.1 Non-Logical Symbols

The non-logical vocabulary of $\mathcal{L}$ includes:

- **Constants:** $a, b, c, d, \dots$

These are individual names. In semantic applications to natural language, constants correspond to proper names such as *Rina*, *Amit*, or *Delhi*. Each constant is intended to denote exactly one individual in the domain of discourse.

- **Variables:** $x, y, z, x_1, x_2, \dots$

Variables range over individuals in the domain. They allow us to express generalizations such as “for all individuals” or “there exists an individual.”

- **Predicate symbols:** $P^1, Q^2, R^3, \dots$

Each predicate has a fixed arity. A one-place predicate (unary) expresses a property; a two-place predicate (binary) expresses a relation between two individuals; a three-place predicate expresses a ternary relation, and so on. For example:

$$Dog(x)$$

expresses a property, while

$$Loves(x, y)$$

expresses a binary relation.

- **Function symbols (optional extension):** $f^1, g^2, \dots$

Function symbols map tuples of individuals to individuals. For example, in arithmetic logic, the successor function $S(x)$ maps a number to its successor. In linguistic semantics, function symbols are less central but can represent operations like “mother-of”:

$$Mother(x)$$

which returns an individual.

Each predicate and function symbol has a fixed arity, meaning the number of arguments it requires is syntactically determined. This ensures structural discipline: writing $Loves(a)$ is syntactically ill-formed if $Loves$ is defined as a two-place predicate.

2.2 Logical Symbols

The logical vocabulary includes:

- Negation: $\neg$

- Conjunction: $\wedge$
- Disjunction: $\vee$
- Conditional: $\rightarrow$
- Biconditional: $\leftrightarrow$
- Universal quantifier: $\forall$
- Existential quantifier: $\exists$
- Identity symbol: $=$
- Parentheses: $()$

Logical symbols contribute structural meaning that remains invariant across interpretations. For example, $\forall$ always expresses universal quantification, regardless of the specific domain.

2.3 Terms

The definition of terms is recursive. Terms are the expressions that denote individuals.

- Every constant is a term.
- Every variable is a term.
- If f is an n -place function symbol and $t_1, \dots, t_n$ are terms, then

$$f(t_1, \dots, t_n)$$

is a term.

Thus, if *Mother* is a one-place function symbol and a is a constant, then

$$Mother(a)$$

is a term. If *Father* is also a function symbol, then

$$Father(Mother(a))$$

is a well-formed term by recursive application.

The recursive definition ensures that arbitrarily complex terms can be constructed while maintaining strict syntactic discipline.

2.4 Atomic Formulas

Atomic formulas are the simplest formulas and are formed by applying predicates to terms.

If P^n is an n -place predicate and $t_1, \dots, t_n$ are terms, then:

$$P(t_1, \dots, t_n)$$

is an atomic formula.

For example:

$$Student(a)$$

$$Loves(a, b)$$

$$Between(a, b, c)$$

Identity statements are also atomic formulas:

$$a = b$$

Atomic formulas express the most basic predications: that an individual has a property, that individuals stand in a relation, or that two individuals are identical.

2.5 Formation Rules for Complex Formulas

The set of well-formed formulas (wffs) is defined recursively:

- Every atomic formula is a formula.
- If ϕ is a formula, then $\neg\phi$ is a formula.
- If ϕ and ψ are formulas, then:

$$(\phi \wedge \psi), \quad (\phi \vee \psi), \quad (\phi \rightarrow \psi), \quad (\phi \leftrightarrow \psi)$$

are formulas.

- If ϕ is a formula and x is a variable, then:

$$\forall x\phi \quad \text{and} \quad \exists x\phi$$

are formulas.

This recursive definition guarantees that every formula has a unique hierarchical structure. Parentheses disambiguate scope and ensure well-formedness. For example:

$$\forall x(Student(x) \rightarrow Passed(x))$$

is well-formed, while:

$$\forall Student(x)$$

is not, because the quantifier must bind a variable.

2.6 Well-Formedness and Structural Hierarchy

One of the most important features of formal syntax is that it generates hierarchical structure rather than mere linear strings. The formula:

$$\neg(P(a) \wedge Q(a))$$

has a tree structure in which negation takes scope over a conjunction. Without parentheses, ambiguity would arise. In natural language, word order may obscure structural relations, but in formal logic, structure is explicitly encoded.

The strict syntactic rules prevent meaningless constructions. For example:

$$(P(a) \wedge)$$

is ill-formed because conjunction requires two formulas. Similarly:

$$\forall(Student(x))$$

is ill-formed because the quantifier must bind a variable explicitly.

2.7 Arity and Structural Discipline

Arity ensures that each predicate takes the correct number of arguments. If *Loves* is binary, then:

$Loves(a, b)$

is well-formed, but:

$Loves(a)$

and

$Loves(a, b, c)$

are not. This structural constraint mirrors natural language subcategorization: verbs require specific argument structures. Thus, logical syntax parallels grammatical valency in linguistics.

2.8 Why Syntactic Precision Matters

The entire semantic enterprise depends on syntactic precision. Truth conditions are defined recursively on syntactic structure. If the syntax were vague, semantic interpretation would collapse. Furthermore, proof theory relies on well-formedness to guarantee that derivations manipulate legitimate formulas.

Predicate logic therefore provides not merely symbolic notation, but a rigorously defined formal language whose structure supports precise semantic and inferential analysis. Its syntax ensures compositional interpretability, structural clarity, and logical discipline—qualities that make it uniquely suitable for modeling natural language meaning.

3. Semantics of Predicate Logic: Model Theory (Extended)

Predicate logic acquires semantic content only once we specify how its expressions are interpreted. Syntax tells us which strings are well-formed; semantics tells us what they mean and under what conditions they are true. In formal semantics, this is achieved through model theory. Model theory provides a mathematically precise definition of truth relative to a structure. It allows us to evaluate logical formulas with respect to a domain of individuals and an interpretation of predicates. In linguistic applications, a model corresponds to a simplified representation of a possible world.

3.1 Structure of a Model

A model $\mathcal{M}$ for a first-order language is an ordered pair:

$$\mathcal{M} = \langle D, I \rangle$$

where:

- D is a non-empty set called the **domain of discourse**.
- I is an **interpretation function**.

The domain D contains the individuals that the language talks about. For example, if we are interpreting sentences about students, the domain might consist of all relevant individuals in a classroom.

The interpretation function assigns denotations to non-logical symbols:

- For each constant c , $I(c) \in D$.
- For each n -place predicate P , $I(P) \subseteq D^n$.
- For each n -place function symbol f , $I(f) : D^n \rightarrow D$.

Thus, predicates denote sets of ordered tuples, and functions denote mappings between individuals.

3.2 Variable Assignments

Unlike constants, variables do not have fixed denotations. Their reference depends on a **variable assignment function** g .

An assignment function:

$$g : Var \rightarrow D$$

maps each variable to an element of the domain.

When evaluating quantified formulas, we modify the assignment. If g is an assignment and $d \in D$, then:

$$g[x \mapsto d]$$

is the assignment identical to g except that it assigns d to x .

Assignments are crucial for defining truth recursively, particularly for quantified expressions.

3.3 Denotation of Terms

The interpretation of terms is defined relative to a model $\mathcal{M}$ and assignment g .

- If t is a constant c , then:

$$c^{\mathcal{M},g} = I(c)$$

- If t is a variable x , then:

$$x^{\mathcal{M},g} = g(x)$$

- If t is a function term $f(t_1, \dots, t_n)$, then:

$$f(t_1, \dots, t_n)^{\mathcal{M},g} = I(f)(t_1^{\mathcal{M},g}, \dots, t_n^{\mathcal{M},g})$$

This recursive clause ensures that even complex nested terms receive well-defined denotations.

3.4 Recursive Definition of Truth (Satisfaction)

Truth is defined recursively using the satisfaction relation:

$$\mathcal{M}, g \models \phi$$

which reads: “ ϕ is satisfied in model $\mathcal{M}$ under assignment g .”

Atomic formulas:

$$\mathcal{M}, g \models P(t_1, \dots, t_n)$$

iff

$$(t_1^{\mathcal{M},g}, \dots, t_n^{\mathcal{M},g}) \in I(P)$$

Identity:

$$\mathcal{M}, g \models (t_1 = t_2)$$

iff

$$t_1^{\mathcal{M},g} = t_2^{\mathcal{M},g}$$

Negation:

$$\mathcal{M}, g \models \neg\phi$$

iff

$$\mathcal{M}, g \not\models \phi$$

Conjunction:

$$\mathcal{M}, g \models \phi \wedge \psi$$

iff both conjuncts are satisfied.

Disjunction and conditionals are defined similarly using classical truth conditions.

Universal quantification:

$$\mathcal{M}, g \models \forall x\phi$$

iff for every $d \in D$,

$$\mathcal{M}, g[x \mapsto d] \models \phi$$

Existential quantification:

$$\mathcal{M}, g \models \exists x\phi$$

iff for some $d \in D$,

$$\mathcal{M}, g[x \mapsto d] \models \phi$$

This recursive structure mirrors the compositional structure of formulas.

3.5 Closed Formulas and Truth in a Model

A formula with no free variables is called a **sentence** or **closed formula**. For closed formulas, truth does not depend on the assignment function.

We write:

$$\mathcal{M} \models \phi$$

if ϕ is true in $\mathcal{M}$ under every assignment.

Thus, truth for sentences becomes a property of the model alone.

3.6 Logical Consequence and Validity

A formula ϕ is **valid** if it is true in every model:

$$\models \phi$$

An argument $\phi_1, \dots, \phi_n \models \psi$ is valid if, in every model where all premises are true, the conclusion is also true.

Logical consequence is therefore defined model-theoretically:

$$\Gamma \models \phi$$

iff every model that satisfies all formulas in Γ also satisfies ϕ .

This definition grounds logical inference in semantic preservation of truth.

3.7 Satisfiability and Consistency

A formula ϕ is **satisfiable** if there exists some model $\mathcal{M}$ such that:

$$\mathcal{M} \models \phi$$

It is **unsatisfiable** if no model makes it true.

A set of formulas Γ is consistent if it is satisfiable. If Γ entails a contradiction, then it is inconsistent.

These notions allow us to evaluate semantic coherence in formal systems.

3.8 Isomorphism and Structural Equivalence

Two models are **isomorphic** if there exists a bijection between their domains preserving interpretation of all symbols. Isomorphic models satisfy exactly the same first-order sentences. This shows that first-order logic cannot distinguish between structures that are structurally identical but differently labeled.

This property demonstrates that logic captures structural relationships rather than accidental naming differences.

3.9 Compactness (Intuitive Discussion)

The Compactness Theorem states:

If every finite subset of a set of formulas is satisfiable, then the whole set is satisfiable.

This meta-theoretic result has profound consequences. It implies that first-order logic cannot enforce finiteness of the domain. If a theory has arbitrarily large finite models, it must also have infinite models.

Although compactness is a deep theorem, its existence highlights the expressive limitations and strengths of first-order logic.

3.10 Connection to Natural Language Semantics

In formal semantics, models are interpreted as simplified possible worlds. The domain contains individuals, and predicates denote properties and relations in that world.

For example:

$$\forall x(Dog(x) \rightarrow Animal(x))$$

is true in a model if the set of dogs is a subset of the set of animals.

Thus, model theory provides:

- A precise definition of truth
- A definition of entailment
- A foundation for compositional interpretation
- A bridge between syntax and meaning

Predicate logic semantics therefore transforms meaning into a mathematically definable relation between language and structure.

3. Semantics of Predicate Logic: Model Theory (Extended)

Predicate logic acquires semantic content only once we specify how its expressions are interpreted. Syntax tells us which strings are well-formed; semantics tells us what they mean and under what conditions they are true. In formal semantics, this is achieved through model theory. Model theory provides a mathematically precise definition of truth relative to a structure. It allows us to evaluate logical formulas with respect to a domain of individuals and an interpretation of predicates. In linguistic applications, a model corresponds to a simplified representation of a possible world.

3.1 Structure of a Model

A model $\mathcal{M}$ for a first-order language is an ordered pair:

$$\mathcal{M} = \langle D, I \rangle$$

where:

- D is a non-empty set called the **domain of discourse**.
- I is an **interpretation function**.

The domain D contains the individuals that the language talks about. For example, if we are interpreting sentences about students, the domain might consist of all relevant individuals in a classroom.

The interpretation function assigns denotations to non-logical symbols:

- For each constant c , $I(c) \in D$.
- For each n -place predicate P , $I(P) \subseteq D^n$.
- For each n -place function symbol f , $I(f) : D^n \rightarrow D$.

Thus, predicates denote sets of ordered tuples, and functions denote mappings between individuals.

3.2 Variable Assignments

Unlike constants, variables do not have fixed denotations. Their reference depends on a **variable assignment function** g .

An assignment function:

$$g : Var \rightarrow D$$

maps each variable to an element of the domain.

When evaluating quantified formulas, we modify the assignment. If g is an assignment and $d \in D$, then:

$$g[x \mapsto d]$$

is the assignment identical to g except that it assigns d to x .

Assignments are crucial for defining truth recursively, particularly for quantified expressions.

3.3 Denotation of Terms

The interpretation of terms is defined relative to a model $\mathcal{M}$ and assignment g .

- If t is a constant c , then:

$$c^{\mathcal{M},g} = I(c)$$

- If t is a variable x , then:

$$x^{\mathcal{M},g} = g(x)$$

- If t is a function term $f(t_1, \dots, t_n)$, then:

$$f(t_1, \dots, t_n)^{\mathcal{M},g} = I(f)(t_1^{\mathcal{M},g}, \dots, t_n^{\mathcal{M},g})$$

This recursive clause ensures that even complex nested terms receive well-defined denotations.

3.4 Recursive Definition of Truth (Satisfaction)

Truth is defined recursively using the satisfaction relation:

$$\mathcal{M}, g \models \phi$$

which reads: “ ϕ is satisfied in model $\mathcal{M}$ under assignment g .”

Atomic formulas:

$$\mathcal{M}, g \models P(t_1, \dots, t_n)$$

iff

$$(t_1^{\mathcal{M},g}, \dots, t_n^{\mathcal{M},g}) \in I(P)$$

Identity:

$$\mathcal{M}, g \models (t_1 = t_2)$$

iff

$$t_1^{\mathcal{M},g} = t_2^{\mathcal{M},g}$$

Negation:

$$\mathcal{M}, g \models \neg\phi$$

iff

$$\mathcal{M}, g \not\models \phi$$

Conjunction:

$$\mathcal{M}, g \models \phi \wedge \psi$$

iff both conjuncts are satisfied.

Disjunction and conditionals are defined similarly using classical truth conditions.

Universal quantification:

$$\mathcal{M}, g \models \forall x \phi$$

iff for every $d \in D$,

$$\mathcal{M}, g[x \mapsto d] \models \phi$$

Existential quantification:

$$\mathcal{M}, g \models \exists x \phi$$

iff for some $d \in D$,

$$\mathcal{M}, g[x \mapsto d] \models \phi$$

This recursive structure mirrors the compositional structure of formulas.

3.5 Closed Formulas and Truth in a Model

A formula with no free variables is called a **sentence** or **closed formula**. For closed formulas, truth does not depend on the assignment function.

We write:

$$\mathcal{M} \models \phi$$

if ϕ is true in $\mathcal{M}$ under every assignment.

Thus, truth for sentences becomes a property of the model alone.

3.6 Logical Consequence and Validity

A formula ϕ is **valid** if it is true in every model:

$$\models \phi$$

An argument $\phi_1, \dots, \phi_n \models \psi$ is valid if, in every model where all premises are true, the conclusion is also true.

Logical consequence is therefore defined model-theoretically:

$$\Gamma \models \phi$$

iff every model that satisfies all formulas in Γ also satisfies ϕ .

This definition grounds logical inference in semantic preservation of truth.

3.7 Satisfiability and Consistency

A formula ϕ is **satisfiable** if there exists some model $\mathcal{M}$ such that:

$$\mathcal{M} \models \phi$$

It is **unsatisfiable** if no model makes it true.

A set of formulas Γ is consistent if it is satisfiable. If Γ entails a contradiction, then it is inconsistent.

These notions allow us to evaluate semantic coherence in formal systems.

3.8 Isomorphism and Structural Equivalence

Two models are **isomorphic** if there exists a bijection between their domains preserving interpretation of all symbols. Isomorphic models satisfy exactly the same first-order sentences. This shows that first-order logic cannot distinguish between structures that are structurally identical but differently labeled.

This property demonstrates that logic captures structural relationships rather than accidental naming differences.

3.9 Compactness (Intuitive Discussion)

The Compactness Theorem states:

If every finite subset of a set of formulas is satisfiable, then the whole set is satisfiable.

This meta-theoretic result has profound consequences. It implies that first-order logic cannot enforce finiteness of the domain. If a theory has arbitrarily large finite models, it must also have infinite models.

Although compactness is a deep theorem, its existence highlights the expressive limitations and strengths of first-order logic.

3.10 Connection to Natural Language Semantics

In formal semantics, models are interpreted as simplified possible worlds. The domain contains individuals, and predicates denote properties and relations in that world.

For example:

$$\forall x(Dog(x) \rightarrow Animal(x))$$

is true in a model if the set of dogs is a subset of the set of animals.

Thus, model theory provides:

- A precise definition of truth
- A definition of entailment
- A foundation for compositional interpretation

- A bridge between syntax and meaning

Predicate logic semantics therefore transforms meaning into a mathematically definable relation between language and structure.

Model theory provides the formal machinery necessary to interpret predicate logic. By specifying domains, interpretation functions, assignment functions, and recursive truth definitions, it allows us to define truth, validity, consequence, satisfiability, and consistency with complete precision. This semantic foundation supports modern formal semantics and underlies much of contemporary linguistic theory.

4. Variables, Binding, and Scope (Extended)

Variables are among the most powerful devices in predicate logic. They allow us to express generality, abstraction, and quantification. However, variables do not have intrinsic reference in the way constants do. Their interpretation depends entirely on whether they are free or bound, and on how they fall within the scope of quantifiers. Understanding variable binding is therefore central to both logic and formal semantics.

4.1 Free and Bound Variables

A variable occurrence in a formula is **bound** if it lies within the scope of a quantifier that binds it. Otherwise, it is **free**.

Formally:

- In $\forall x\phi$, every occurrence of x in ϕ is bound by $\forall x$.
- In $\exists x\phi$, every occurrence of x in ϕ is bound by $\exists x$.

For example:

$$\forall x(Student(x) \rightarrow Passed(x))$$

Here, both occurrences of x are bound by the universal quantifier.

In contrast:

$$Student(x) \rightarrow Passed(x)$$

contains two free occurrences of x . This formula is not a sentence; its truth depends on the assignment function. A formula containing free variables is often called an **open formula**.

A formula with no free variables is called a **closed formula** or a **sentence**. Only closed formulas can be evaluated as true or false in a model independently of assignments.

4.2 Scope of Quantifiers

The **scope** of a quantifier is the formula over which it has syntactic authority. In:

$$\forall x(P(x) \rightarrow Q(x))$$

the scope of $\forall x$ is the entire parenthesized formula.

Parentheses are crucial because they determine hierarchical structure. Consider:

$$\forall x P(x) \rightarrow Q(x)$$

Without parentheses, this formula is ambiguous. By convention, quantifiers bind as far right as possible, so this would be read as:

$$(\forall x P(x)) \rightarrow Q(x)$$

which is very different from:

$$\forall x(P(x) \rightarrow Q(x))$$

In the first, the quantifier applies only to $P(x)$; in the second, it applies to the whole conditional.

Scope thus determines semantic interpretation. Logical structure, not linear order, determines scope relations.

4.3 Binding vs Reference

Binding is a syntactic relation; reference is a semantic relation.

A bound variable does not refer to a fixed individual. Instead, it ranges over elements of the domain under quantification.

For example:

$$\forall x \text{Dog}(x)$$

does not assert that a particular individual is a dog. It asserts that every individual in the domain satisfies the property *Dog*.

In contrast, a constant such as *a* directly refers to a specific individual:

$$\text{Dog}(a)$$

asserts that a particular individual is a dog.

Thus, binding introduces variable dependence rather than reference.

4.4 Substitution and Variable Capture

Substitution must be handled carefully to avoid **variable capture**. Suppose we attempt to substitute a term *t* for a variable *x* in a formula $\phi(x)$. If *t* contains variables that become accidentally bound after substitution, unintended meaning results.

Example:

$$\forall x P(x, y)$$

Suppose we substitute *x* for *y*. Naively substituting gives:

$$\forall x P(x, x)$$

But now the second occurrence of *x* has become bound, whereas originally *y* was free. This changes the meaning drastically.

To avoid such capture, substitution rules must ensure that bound variables are renamed when necessary.

4.5 Alpha-Conversion

Bound variable names are arbitrary. We can systematically rename them without changing meaning. This process is called **alpha-conversion**.

For example:

$$\forall x P(x)$$

is logically equivalent to:

$$\forall y P(y)$$

provided y does not already occur free in the formula.

Alpha-conversion ensures that logical content does not depend on particular variable names.

4.6 Nested Quantifiers and Binding Structure

Consider:

$$\forall x \exists y \text{Loves}(x, y)$$

Here:

- x is bound by the universal quantifier.
- y is bound by the existential quantifier.

The scope relations are hierarchical. The existential quantifier lies within the scope of the universal. This means:

For every individual x , there exists some individual y (possibly depending on x) such that x loves y .

Now compare:

$$\exists y \forall x \text{Loves}(x, y)$$

This says there exists a single individual loved by everyone.

Thus, variable binding structure directly determines semantic interpretation.

4.7 Quantifier Scope Ambiguity in Natural Language

Natural language often displays scope ambiguity. Consider:

Every student read a book.

This sentence corresponds to two logical forms:

$$\forall x \exists y \text{Read}(x, y)$$

and

$$\exists y \forall x \text{Read}(x, y)$$

The first interpretation allows different books for different students; the second requires one book read by all students.

The ambiguity arises because different logical structures correspond to the same surface string. Logical form makes scope relations explicit.

4.8 Shadowing and Variable Independence

In:

$$\forall x (\exists x P(x))$$

the inner quantifier shadows the outer one. The inner x is a different bound variable. To avoid confusion, we usually rename:

$$\forall x (\exists y P(y))$$

This makes clear that the two quantifiers bind independent variables.

Shadowing shows again that variable names are merely placeholders; structural binding relations matter, not the symbols themselves.

4.9 Binding and Logical Dependence

Binding introduces dependency relations. In:

$$\forall x \exists y R(x, y)$$

the value of y may depend on x . This expresses a functional dependency.

In contrast:

$$\exists y \forall x R(x, y)$$

expresses independence: one y works for all x .

These logical distinctions are crucial in semantic theory, particularly in the analysis of pronouns, anaphora, and quantifier interactions.

4.10 Linguistic Binding and Anaphora

Variable binding in logic parallels pronoun binding in natural language.

Sentence:

Every student thinks he is intelligent.

Logical representation:

$$\forall x (Student(x) \rightarrow Thinks(x, Intelligent(x)))$$

The pronoun “he” corresponds to a bound variable dependent on the quantifier.

Thus, logical binding provides a formal model for linguistic anaphora.

Variable binding is the mechanism that allows logic to express generality and dependency. The distinction between free and bound variables determines whether a formula is open or closed. Scope determines which occurrences are governed by which quantifiers. Alpha-conversion ensures that bound variable names are inessential. Proper management of substitution avoids capture. Finally, binding structures correspond directly to semantic distinctions in natural language, especially in quantification and anaphora.

Understanding variable binding is therefore foundational not only for logic but for formal semantics as a whole.

5. Identity and Definite Descriptions (Extended)

Identity and definite descriptions occupy a central place in formal semantics because they connect reference, quantification, and logical structure in subtle ways. While identity appears to be a simple notion—two expressions refer to the same object—its logical behavior has deep consequences. Definite descriptions, expressions of the form “the F ,” raise foundational questions about existence, uniqueness, and reference. Predicate logic provides precise tools for analyzing both phenomena.

5.1 The Logic of Identity

The identity symbol “=” is treated as a logical constant in first-order logic. It expresses numerical identity between individuals. The formula:

$$a = b$$

is true in a model $\mathcal{M}$ if and only if:

$$I(a) = I(b)$$

That is, the two constants denote the same element of the domain.

Identity satisfies three fundamental logical properties:

Reflexivity:

$$\forall x(x = x)$$

Symmetry:

$$\forall x \forall y (x = y \rightarrow y = x)$$

Transitivity:

$$\forall x \forall y \forall z ((x = y \wedge y = z) \rightarrow x = z)$$

Together these properties ensure that identity forms an equivalence relation on the domain.

5.2 Substitutivity of Identicals (Leibniz's Law)

One of the most powerful principles governing identity is the substitutivity of identicals, often called Leibniz's Law:

If $a = b$, then any property true of a is also true of b .

Formally:

$$a = b \rightarrow (P(a) \leftrightarrow P(b))$$

More generally:

$$x = y \rightarrow (\phi(x) \leftrightarrow \phi(y))$$

provided substitution does not occur inside intensional contexts.

For example, if:

$$\textit{MorningStar} = \textit{EveningStar}$$

and

$$\textit{Bright}(\textit{MorningStar})$$

then we may infer:

$$\textit{Bright}(\textit{EveningStar})$$

However, in intensional contexts such as belief reports, substitutivity may fail:

$$\textit{John believes that MorningStar is bright}$$

does not necessarily imply:

$$\textit{John believes that EveningStar is bright}$$

This shows that identity behaves differently in intensional environments, a topic addressed in intensional semantics.

5.3 Definite Descriptions: The Russellian Analysis

Definite descriptions are expressions of the form:

the F

For example:

The president resigned.

Bertrand Russell proposed that definite descriptions are not simple referring expressions but quantified structures. According to Russell, “The F is G ” means:

1. There exists at least one F .
2. There exists at most one F .
3. That F is G .

Formally:

$$\exists x(F(x) \wedge \forall y(F(y) \rightarrow y = x) \wedge G(x))$$

This formula contains three components:

- **Existence:** $\exists x F(x)$
- **Uniqueness:** $\forall y(F(y) \rightarrow y = x)$
- **Predication:** $G(x)$

Thus, “The president resigned” is analyzed as:

$$\exists x(President(x) \wedge \forall y(President(y) \rightarrow y = x) \wedge Resigned(x))$$

The logical structure reveals that definite descriptions involve quantification rather than simple naming.

5.4 Failure of Reference and Non-Existence

Consider:

The king of France is bald.

If there is no king of France, then the existence condition fails. Under Russell’s analysis, the sentence is false because the existential clause is false.

Formally:

$$\exists x(KingOfFrance(x) \wedge \dots)$$

is false if no such x exists.

This avoids the paradox of treating “the king of France” as a referring term without referent. Instead, the description contributes quantificational structure.

5.5 Strawson’s Critique

Peter Strawson argued that sentences like “The king of France is bald” do not assert falsehood when the referent does not exist; rather, they presuppose existence. If the presupposition fails, the sentence lacks a truth value.

Russell’s approach treats existence as part of the asserted content; Strawson treats it as a presupposition.

In modern formal semantics, presupposition is modeled separately, but Russell’s logical analysis remains foundational for understanding definite descriptions compositionally.

5.6 Definite Descriptions and Scope

Definite descriptions can interact with other quantifiers in scope-sensitive ways.

Consider:

Every student read the book.

Logical form:

$$\forall x(Student(x) \rightarrow Read(x, b))$$

where b denotes the uniquely identifiable book.

Now consider:

The teacher praised every student.

Logical form:

$$\exists x(Teacher(x) \wedge \forall y(Teacher(y) \rightarrow y = x) \wedge \forall z(Student(z) \rightarrow Praised(x, z)))$$

The description “the teacher” may take wide scope over the universal quantifier.

Scope interactions demonstrate that definite descriptions behave like quantified expressions rather than simple names.

5.7 Identity and Uniqueness

The uniqueness clause in Russell’s analysis relies crucially on identity:

$$\forall y(F(y) \rightarrow y = x)$$

This states that any object satisfying F must be identical to x .

Without identity, uniqueness could not be formally expressed.

Thus, identity plays a central role in defining definite descriptions.

5.8 Definite Descriptions in Intensional Contexts

In modal or belief contexts, definite descriptions may exhibit scope ambiguities.

John believes that the winner is dishonest.

This can mean:

- There is a particular individual who is the winner, and John believes that individual is dishonest (de re reading).
- John believes that whoever satisfies the description “winner” is dishonest (de dicto reading).

Logical representations distinguish these readings by varying scope relative to belief operators.

Identity provides the logical machinery for expressing sameness and uniqueness. It satisfies reflexivity, symmetry, and transitivity, and supports substitution in extensional contexts. Definite descriptions, analyzed through Russell’s theory, are quantificational structures asserting existence, uniqueness, and predication. Their interaction with scope, presupposition, and intensionality reveals deep connections between logic and natural language meaning.

Through the formal tools of predicate logic, identity and definite descriptions become precisely analyzable components of semantic structure.

6. Inference and Validity (Extended)

Inference lies at the heart of logic. While syntax determines which expressions are well-formed and semantics determines their truth conditions, inference concerns the preservation of truth from premises to conclusions. A system of logic is not merely a symbolic language but also a theory of correct reasoning. In formal semantics, inference is crucial because meaning determines patterns of entailment. If a speaker understands the meaning of a sentence, they also understand what follows from it logically. Predicate logic provides a precise framework for analyzing such inferential relations.

6.1 Logical Consequence (Semantic Definition)

The most fundamental notion in logic is **logical consequence**. A formula ψ is a logical consequence of a set of premises Γ if, in every model in which all members of Γ are true, ψ is also true.

Formally:

$$\Gamma \models \psi$$

means:

For every model $\mathcal{M}$, if $\mathcal{M} \models \phi$ for all $\phi \in \Gamma$, then $\mathcal{M} \models \psi$.

Logical consequence is therefore defined model-theoretically. It depends on truth preservation across all possible interpretations.

Example:

$$\begin{array}{c} \forall x (Human(x) \rightarrow Mortal(x)) \\ Human(a) \end{array}$$

entails:

$$Mortal(a)$$

This inference is valid because in any model where the premises are true, the conclusion must also be true.

6.2 Validity

A formula ϕ is **valid** if it is true in every model:

$$\models \phi$$

Validity is a property of individual formulas. Logical consequence is a relation between premises and conclusion.

Example of a valid formula:

$$\forall x (P(x)) \rightarrow P(a)$$

This is valid because if every individual has property P , then any specific individual must also have property P .

A formula is **satisfiable** if it is true in at least one model. A formula is a **contradiction** if it is false in every model.

Thus we distinguish:

- Valid (true in all models)
- Satisfiable (true in at least one model)
- Unsatisfiable (true in none)

6.3 Proof-Theoretic Inference

While semantic consequence is defined using models, proof theory provides syntactic rules for deriving conclusions from premises. A derivation system attempts to mirror semantic consequence through formal rules.

We write:

$$\Gamma \vdash \psi$$

to mean that ψ is derivable from Γ using inference rules.

A logical system is:

- **Sound** if $\Gamma \vdash \psi$ implies $\Gamma \models \psi$.
- **Complete** if $\Gamma \models \psi$ implies $\Gamma \vdash \psi$.

Soundness ensures that derivations do not produce false conclusions from true premises. Completeness ensures that all semantically valid inferences are derivable.

6.4 Basic Inference Rules

Modus Ponens:

$$\begin{array}{c} \phi \\ \phi \rightarrow \psi \\ \hline \therefore \psi \end{array}$$

Universal Instantiation:

$$\begin{array}{c} \forall x \phi(x) \\ \hline \therefore \phi(a) \end{array}$$

Existential Introduction:

$$\phi(a)$$

$$\therefore \exists x\phi(x)$$

Universal Generalization (with restriction):

If $\phi(x)$ has been derived without assumptions about x , then:

$$\forall x\phi(x)$$

Quantifier reasoning introduces special restrictions to avoid invalid generalizations.

6.5 Countermodels and Invalidity

To show that an argument is invalid, we construct a countermodel—a model in which the premises are true but the conclusion is false.

Example:

Premises:

$$\exists xDog(x)$$

Conclusion:

$$\forall xDog(x)$$

Countermodel:

Domain: $\{a, b\}$

Interpretation:

$$Dog = \{a\}$$

Then $\exists xDog(x)$ is true, but $\forall xDog(x)$ is false.

Thus the inference is invalid.

Countermodels are powerful tools because a single counterexample suffices to refute validity.

6.6 Logical Equivalence

Two formulas ϕ and ψ are logically equivalent if:

$$\models \phi \leftrightarrow \psi$$

This means they are true in exactly the same models.

Example:

$$\neg \forall x P(x)$$

is logically equivalent to:

$$\exists x \neg P(x)$$

These equivalences allow transformations between logically equivalent forms.

6.7 Tautologies and Contradictions

A tautology is a formula true in every model.

Example (propositional level):

$$P \vee \neg P$$

A contradiction is false in every model:

$$P \wedge \neg P$$

In predicate logic, contradictions may involve quantifiers, such as:

$$\forall x P(x) \wedge \exists x \neg P(x)$$

These concepts help classify formulas semantically.

6.8 Entailment in Natural Language

Logical consequence corresponds to semantic entailment in language.

Sentence:

Every dog barked.

entails:

Some dog barked.

Formalization:

$$\forall x(Dog(x) \rightarrow Barked(x))$$

entails:

$$\exists x(Dog(x) \wedge Barked(x))$$

provided the domain contains at least one dog.

Inference patterns thus reveal structural aspects of meaning.

6.9 Entailment vs Presupposition

Entailment differs from presupposition. If sentence A entails B , then whenever A is true, B must be true.

Presupposition, however, remains constant under negation.

Example:

The king of France is bald.

presupposes:

There is a king of France.

Even its negation presupposes existence.

Logical inference operates at the level of asserted content, whereas presupposition concerns background assumptions.

Inference and validity formalize correct reasoning. Logical consequence is defined model-theoretically as truth preservation across all models. Validity is truth in every model. Proof systems attempt to capture semantic consequence syntactically. Countermodels demonstrate

invalidity. Logical equivalence, satisfiability, tautology, and contradiction classify formulas semantically. Finally, formal inference mirrors patterns of entailment in natural language, making logic an indispensable tool in semantic theory.

7. Type Theory and Compositional Semantics (Extended)

Type theory provides the formal backbone for compositional semantics. While predicate logic supplies the machinery for representing relations and quantification, type theory ensures that semantic composition proceeds in a structurally disciplined way. The central idea is that meanings belong to different semantic categories (types), and composition is possible only when types match appropriately. In this way, type theory prevents semantic incoherence and enforces systematic interpretation.

7.1 Motivation for Type Theory

Natural language expressions differ in semantic complexity. Proper names denote individuals, sentences denote truth values, verbs denote relations, and quantifiers denote higher-order functions. Without a type system, nothing would prevent us from combining semantically incompatible expressions. For example, if “Rina” denotes an individual and “true” denotes a truth value, applying one to the other would be meaningless. Type theory prevents such category mistakes by classifying semantic objects into distinct kinds.

The principle of compositionality states:

The meaning of a complex expression is determined by the meanings of its parts and the way they are combined.

Type theory provides the formal constraints that make this principle precise.

7.2 Basic Semantic Types

The simplest semantic types are:

$$\begin{aligned} e &= \text{type of entities} \\ t &= \text{type of truth values} \end{aligned}$$

An expression of type e denotes an individual in the domain. An expression of type t denotes a truth value (True or False).

Examples:

- Proper name: *Rina* type e
- Sentence: *Rina laughs* type t

7.3 Functional Types

If α and β are types, then:

$$\langle \alpha, \beta \rangle$$

is the type of functions from objects of type α to objects of type β .

For example:

$$\langle e, t \rangle$$

is the type of one-place predicates (properties). A noun like “student” denotes a function from individuals to truth values:

$$student : D \rightarrow \{0, 1\}$$

Thus:

$$student : \langle e, t \rangle$$

A transitive verb has type:

$$\langle e, \langle e, t \rangle \rangle$$

because it takes an object and returns a function awaiting a subject.

7.4 Type Assignment to Syntactic Categories

The mapping from syntax to semantics assigns types to syntactic categories.

- Noun (N): $\langle e, t \rangle$
- Intransitive verb (VP): $\langle e, t \rangle$
- Transitive verb (V): $\langle e, \langle e, t \rangle \rangle$
- Proper name (NP): e
- Sentence (S): t

Example:

Rina laughs

Semantic derivation:

laughs : $\langle e, t \rangle$
Rina : e

Applying the predicate to the argument yields a truth value:

laughs(*Rina*) : t

7.5 Function Application

The fundamental rule of semantic composition is **Function Application**:

If α has type $\langle \sigma, \tau \rangle$ and β has type σ , then:

$\alpha(\beta)$

has type τ .

This rule ensures type matching.

Example:

Loves(*Amit*)

where:

$$Loves : \langle e, \langle e, t \rangle \rangle$$

First apply to object:

$$Loves(Amit) : \langle e, t \rangle$$

Then apply to subject:

$$Loves(Amit)(Rina)$$

which yields type t .

7.6 Lambda Abstraction and Functional Representation

To represent partial application formally, we use lambda abstraction.

Example:

$$\lambda x. Laugh(x)$$

This expression has type $\langle e, t \rangle$.

For transitive verbs:

$$\lambda y \lambda x Loves(x, y)$$

This function takes an object y , then a subject x .

Applying:

$$(\lambda y \lambda x Loves(x, y))(Amit) = \lambda x Loves(x, Amit)$$

Then applying to Rina:

$$Loves(Rina, Amit)$$

Lambda abstraction therefore formalizes compositional interpretation.

7.7 Higher-Order Types

Quantifiers introduce higher-order types. Instead of denoting individuals, quantified NPs denote sets of properties.

For example:

Every student

denotes:

$$\lambda P.\forall x(Student(x) \rightarrow P(x))$$

This has type:

$$\langle\langle e, t \rangle, t \rangle$$

It takes a predicate P and returns a truth value.

Sentence:

Every student laughed

Derivation:

$$Every\ student(\lambda x Laugh(x))$$

which reduces to:

$$\forall x(Student(x) \rightarrow Laugh(x))$$

Thus generalized quantifiers require higher-order types.

7.8 Type Mismatches and Type Shifting

Sometimes semantic types do not align straightforwardly. For example, proper names are type e , but quantified NPs are type $\langle\langle e, t \rangle, t\rangle$.

To achieve uniformity, type shifting rules are introduced. A proper name may be lifted:

$$Rina \rightsquigarrow \lambda P.P(Rina)$$

Thus Rina becomes a generalized quantifier.

This allows uniform treatment of NPs in subject position.

7.9 Compositional Derivation Example

Consider:

$$Every\ student\ loves\ Amit$$

Lexical entries:

$$student : \lambda x Student(x)$$

$$loves : \lambda y \lambda x Loves(x, y)$$

$$Amit : a$$

Step 1: Combine verb and object:

$$loves(Amit) = \lambda x Loves(x, a)$$

Step 2: Combine quantifier:

$$(\lambda P. \forall x (Student(x) \rightarrow P(x)))(\lambda x Loves(x, a))$$

Result:

$$\forall x (Student(x) \rightarrow Loves(x, a))$$

The derivation proceeds strictly via function application.

7.10 Compositionality as Constraint

Type theory enforces compositionality because:

- Every syntactic combination must correspond to a type-valid semantic operation.
- Ill-typed expressions are excluded.
- Semantic derivations follow structural hierarchy.

Thus, type theory ensures that semantic interpretation mirrors syntactic structure.

Type theory classifies meanings into structured categories and defines how they combine. Basic types (e , t) serve as foundations, while functional types allow complex expressions. Lambda abstraction encodes functions, generalized quantifiers require higher-order types, and type shifting resolves mismatches. Through these mechanisms, compositional semantics becomes a mathematically precise system in which structure determines meaning.

8. Lambda Abstraction (Extended)

Lambda abstraction is one of the most powerful tools in formal semantics. While predicate logic provides the structure for representing relations and quantification, lambda calculus provides the mechanism for representing functions and for computing semantic composition. Without lambda abstraction, compositional interpretation would require cumbersome meta-level descriptions. Lambda abstraction allows us to treat meanings as mathematical functions inside the object language itself.

8.1 Motivation for Lambda Abstraction

In natural language, predicates combine with arguments. Consider:

Rina laughs

We can represent this as:

Laugh(Rina)

But consider the verb phrase:

laughs

It denotes something that awaits an argument. Intuitively, it expresses the property of being someone who laughs. Thus it must denote a function from individuals to truth values.

Lambda abstraction provides the formal notation for representing such functions:

$\lambda x.Laugh(x)$

This expression denotes a function that maps any individual x to the truth value of $Laugh(x)$.

Thus lambda abstraction allows us to represent incomplete expressions (like verb phrases) as functions.

8.2 Syntax of Lambda Expressions

If x is a variable and ϕ is an expression, then:

$\lambda x.\phi$

is a lambda expression.

Intuitively:

$\lambda x.\phi$

means: the function that takes an argument x and returns ϕ .

The variable x is bound within the scope of the lambda operator.

Example:

$\lambda x.Student(x)$

denotes the property of being a student.

8.3 Free and Bound Variables in Lambda Expressions

In:

$$\lambda x.Loves(x, y)$$

the variable x is bound by the lambda operator, while y is free.

A lambda expression denotes a function whose value depends only on the bound variable(s).

Free variables remain dependent on external context.

8.4 Function Application and Beta Reduction

Lambda expressions combine with arguments through **function application**. The fundamental computational rule is called **beta reduction**:

$$(\lambda x.\phi)(a) \Rightarrow \phi[a/x]$$

This means: substitute a for every free occurrence of x in ϕ .

Example:

$$(\lambda x.Laugh(x))(Rina)$$

reduces to:

$$Laugh(Rina)$$

More complex example:

$$(\lambda y \lambda x.Loves(x, y))(Amit)$$

reduces to:

$$\lambda x.Loves(x, Amit)$$

Applying again:

$$(\lambda x \text{Loves}(x, \text{Amit}))(\text{Rina})$$

reduces to:

$$\text{Loves}(\text{Rina}, \text{Amit})$$

Thus beta reduction formalizes semantic composition.

8.5 Alpha Conversion

The names of bound variables are arbitrary. We may rename them without changing meaning. This is called **alpha conversion**.

$$\lambda x.P(x)$$

is equivalent to:

$$\lambda y.P(y)$$

provided y does not occur free in the expression.

Alpha conversion prevents confusion during substitution and avoids variable capture.

8.6 Eta Conversion

Eta conversion expresses extensional equivalence of functions.

$$\lambda x.f(x)$$

is equivalent to:

$$f$$

provided x does not occur free in f .

Eta conversion reflects the idea that a function that simply applies another function is identical to that function.

8.7 Typed Lambda Calculus

In semantic theory, lambda calculus is typed.

If x has type σ and ϕ has type τ , then:

$$\lambda x.\phi$$

has type:

$$\langle\sigma, \tau\rangle$$

For example:

$$\lambda x.Laugh(x)$$

has type:

$$\langle e, t\rangle$$

because x is of type e and $Laugh(x)$ is of type t .

Type discipline ensures that only meaningful function applications occur.

8.8 Lambda Abstraction in Verb Phrase Semantics

Consider:

$$Rina\ loves\ Amit$$

Lexical entries:

$$loves = \lambda y\lambda xLoves(x, y)$$

Step 1: Apply to object:

$$(\lambda y \lambda x \text{Loves}(x, y))(Amit)$$

Beta reduction gives:

$$\lambda x \text{Loves}(x, Amit)$$

This is the meaning of the verb phrase “loves Amit.”

Step 2: Apply to subject:

$$(\lambda x \text{Loves}(x, Amit))(Rina)$$

Beta reduction gives:

$$\text{Loves}(Rina, Amit)$$

Thus lambda abstraction allows hierarchical composition.

8.9 Lambda Abstraction and Quantifiers

Consider:

$$\textit{Every student laughed}$$

Quantifier meaning:

$$\textit{Every} = \lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x))$$

Student:

$$\lambda x \textit{Student}(x)$$

Laughed:

$$\lambda x Laugh(x)$$

Composition:

$$(\lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x))) (\lambda x Student(x))$$

reduces to:

$$\lambda Q. \forall x (Student(x) \rightarrow Q(x))$$

Applying to $\lambda x Laugh(x)$ yields:

$$\forall x (Student(x) \rightarrow Laugh(x))$$

Thus lambda abstraction enables higher-order functional application.

8.10 Multiple Abstraction and Currying

Functions of multiple arguments are represented through nested abstractions (currying).

Instead of:

$$f(x, y)$$

we represent:

$$\lambda x \lambda y f(x, y)$$

This transforms multi-place relations into chains of unary functions, aligning with functional type theory.

8.11 Lambda Abstraction and Relative Clauses

Consider:

students who laughed

Semantic representation:

$$\lambda x(Student(x) \wedge Laugh(x))$$

The relative clause modifies the noun via predicate conjunction. Lambda abstraction ensures that modification preserves type $\langle e, t \rangle$.

8.12 Computational Perspective

Lambda calculus is not merely notation but a computational system. Semantic derivations correspond to reductions in a formal calculus. This makes compositional semantics algorithmic in nature, which is crucial for computational linguistics and formal grammar implementations.

Lambda abstraction provides the formal mechanism for representing functions within semantic theory. Through beta reduction, it models function application; through alpha conversion, it ensures variable independence; through typed lambda calculus, it enforces compositional correctness. Lambda abstraction enables representation of verb phrases, quantifiers, relative clauses, and higher-order constructions. It transforms semantic interpretation into a systematic computational process governed by precise mathematical rules.

9. Generalized Quantifiers (Extended)

The classical quantifiers of first-order logic—universal ($\forall$) and existential ($\exists$)—are sufficient for representing simple statements of generality. However, natural language contains a much richer system of quantificational expressions: “every,” “some,” “most,” “many,” “few,” “no,” “exactly three,” and so on. These expressions do not behave uniformly as individual-denoting terms. Instead, they operate at a higher semantic level. Generalized Quantifier Theory provides a formal framework for representing these expressions within compositional semantics.

9.1 Motivation

Consider the sentence:

Every student laughed.

In classical predicate logic, we represent this as:

$$\forall x(Student(x) \rightarrow Laugh(x))$$

But from a compositional perspective, this analysis hides the internal structure of the determiner “every.” If “student” denotes a property $\lambda x Student(x)$, then the determiner must combine with that property to form a noun phrase. That noun phrase then combines with the verb phrase.

This requires a higher-order interpretation of determiners.

9.2 Formal Definition of Generalized Quantifiers

Let D be a domain of individuals. A generalized quantifier over D is a set of subsets of D .

Formally, a generalized quantifier is a function:

$$Q : \mathcal{P}(D) \rightarrow \{0, 1\}$$

or equivalently, an element of:

$$\mathcal{P}(\mathcal{P}(D))$$

In type-theoretic terms, noun phrases have type:

$$\langle \langle e, t \rangle, t \rangle$$

They take a predicate (type $\langle e, t \rangle$) and return a truth value.

9.3 Determiners as Higher-Order Functions

The determiner “every” is interpreted as:

$$\lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x))$$

Here:

- P corresponds to the noun meaning.
- Q corresponds to the verb phrase meaning.

Type:

$$\langle\langle e, t \rangle, \langle\langle e, t \rangle, t \rangle\rangle$$

After combining with a noun, the result is a generalized quantifier of type $\langle\langle e, t \rangle, t \rangle$.

9.4 Compositional Derivation Example

Sentence:

Every student laughed.

Lexical entries:

$$\begin{aligned} student &= \lambda x Student(x) \\ laughed &= \lambda x Laugh(x) \\ every &= \lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x)) \end{aligned}$$

Step 1: Combine determiner and noun:

$$(\lambda P \lambda Q. \forall x (P(x) \rightarrow Q(x))) (\lambda x Student(x))$$

reduces to:

$$\lambda Q. \forall x (Student(x) \rightarrow Q(x))$$

Step 2: Apply to VP:

$$(\lambda Q. \forall x (Student(x) \rightarrow Q(x))) (\lambda x Laugh(x))$$

reduces to:

$$\forall x(Student(x) \rightarrow Laugh(x))$$

Thus the quantifier meaning is derived compositionally.

9.5 Existential Quantifiers

“Some” is interpreted as:

$$\lambda P \lambda Q. \exists x(P(x) \wedge Q(x))$$

Thus:

Some student laughed

derives:

$$\exists x(Student(x) \wedge Laugh(x))$$

9.6 Conservativity

A fundamental property of natural language determiners is **conservativity**.

A determiner Q is conservative iff:

$$Q(A)(B) \text{ is true iff } Q(A)(A \cap B)$$

Intuitively, when evaluating “Q A are B,” we only need to consider individuals in A .

Example:

Every student laughed

is true iff every student is in the intersection of students and laughers.

All natural language determiners are conservative.

9.7 Monotonicity

Generalized quantifiers exhibit monotonicity properties.

Upward Monotone in Second Argument:

A quantifier Q is upward monotone if:

$$Q(A)(B) \text{ and } B \subseteq C \Rightarrow Q(A)(C)$$

Example:

“Some” is upward monotone.

If some students laughed, and laughing implies smiling, then some students smiled.

Downward Monotone:

“No” is downward monotone.

If no students laughed, and laughing implies singing, then no students sang.

Monotonicity properties are crucial for inference patterns in natural language.

9.8 Strong and Weak Quantifiers

Quantifiers are often classified as:

- **Strong:** every, most, all
- **Weak:** some, many, a few

Strong quantifiers typically presuppose non-emptiness and resist existential constructions like:

There are every students.

Weak quantifiers allow such constructions:

There are some students.

This distinction relates to semantic type and discourse function.

9.9 Proportional Quantifiers

Quantifiers such as “most” require comparison between cardinalities.

Most students laughed

can be interpreted as:

$$|Student \cap Laugh| > |Student - Laugh|$$

Such quantifiers cannot be expressed directly in first-order logic. They require cardinality comparisons, showing that natural language quantification exceeds simple logical quantifiers.

9.10 Scope Ambiguity

Quantifier interaction produces scope ambiguity.

Sentence:

Every student read a book.

Two logical forms:

$$\forall x \exists y Read(x, y)$$

and

$$\exists y \forall x Read(x, y)$$

In generalized quantifier terms, different scope orders correspond to different orders of functional application.

Scope ambiguity demonstrates the flexibility of quantifier interpretation.

9.11 Quantifier Raising (Informal)

In syntactic theory, scope differences are sometimes explained through Quantifier Raising (QR), a covert movement operation that changes hierarchical structure at Logical Form. Although QR is syntactic, generalized quantifier semantics provides the compositional mechanism that interprets such structures.

Generalized Quantifier Theory extends classical logic to capture the richness of natural language determiners. Noun phrases are interpreted as higher-order functions of type $\langle\langle e, t \rangle, t\rangle$. Determiners combine with noun meanings to form generalized quantifiers, which then apply to predicate meanings. Properties such as conservativity and monotonicity characterize natural language quantification. Scope interactions and proportional quantifiers reveal the expressive power of this framework. Generalized quantifiers therefore constitute a central component of modern compositional semantics.

10. Intensionality and Possible Worlds

Extensional semantics evaluates truth relative to one world. Intensional semantics evaluates truth relative to a world parameter.

A model becomes:

$$\mathcal{M} = \langle W, D, I \rangle$$

Modal operator:

$$\Box\phi$$

is true at world w if ϕ is true at all accessible worlds.

Belief contexts:

$$Believes(j, \phi)$$

require evaluation relative to belief-accessible worlds.

This explains substitution failure in opaque contexts.

11. Formal Properties

Soundness: If $\vdash \phi$, then $\models \phi$.

Completeness: If $\models \phi$, then $\vdash \phi$.

Compactness: If every finite subset of a set of formulas is satisfiable, then the whole set is satisfiable.

These meta-theorems establish the robustness of first-order logic as a foundation for semantic theory.